Descrição da estratégia escolhida para o Saga Pattern

A estratégia escolhida para a Officyna é a **Saga por coreografia**, orientada a eventos. Nessa abordagem, não há um único componente central responsável por controlar todas as etapas da transação distribuída. Cada microsserviço reage a eventos que lhe dizem respeito, executa sua própria transação local e publica um novo evento para que os demais participantes continuem o fluxo.

Essa escolha é adequada ao projeto porque os domínios de Ordem de Serviço, execução, estoque e faturamento possuem responsabilidades independentes e podem ser processados de forma assíncrona. Assim, evita-se uma transação única e distribuída entre diferentes bancos e serviços, que seria mais complexa, mais frágil e contrária à autonomia esperada em uma arquitetura de microsserviços.

Fluxo principal da Saga

1. O `officyna-os-service`, responsável pelo ciclo de vida da Ordem de Serviço, inicia o fluxo ao publicar um evento de diagnóstico para a fila SQS.
2. O `officyna-execution-production` consome esse evento, cria o snapshot operacional da OS e conduz as atividades de diagnóstico, mão de obra, catálogo e estoque.
3. Ao finalizar o diagnóstico e solicitar aprovação, o serviço de execução reserva os insumos necessários e publica os eventos correspondentes. O fluxo segue para a geração de orçamento.
4. O `officyna-billing-service` consome o evento de geração de pagamento, calcula os valores de mão de obra, peças, ISS e ICMS, persiste o orçamento em sua própria base PostgreSQL e publica o resultado.
5. Após a aprovação do cliente, o serviço de faturamento recebe a solicitação de checkout, integra-se ao Mercado Pago e aguarda a confirmação do pagamento por webhook.
6. Quando o pagamento é confirmado, o serviço de faturamento publica o evento `eventOrderPaid` em um tópico SNS. O `officyna-os-service` e o `officyna-execution-production` podem consumir esse evento de forma independente, atualizando seus respectivos estados e permitindo a continuidade da execução.
7. Ao iniciar e finalizar a execução, o serviço de produção publica eventos de atualização. O serviço de OS permanece como dono do estado oficial da Ordem de Serviço, enquanto o serviço de execução mantém somente seu snapshot operacional.

Compensações

Como não existe transação distribuída entre MongoDB/DocumentDB, PostgreSQL, SQS, SNS e Mercado Pago, a consistência é alcançada de forma eventual. Para tratar falhas, são aplicadas ações compensatórias.

No fluxo de diagnóstico, se uma mensagem não puder ser processada após as tentativas previstas, ela é encaminhada para uma DLQ. O serviço de execução consome a compensação e retorna o snapshot local para o estado `RECEBIDA`.

No fluxo de aprovação, se o processamento falhar após a reserva de estoque, a compensação libera os insumos reservados e devolve o fluxo para `EM_DIAGNOSTICO`. Dessa forma, evita-se que estoque permaneça indevidamente bloqueado por uma OS que não avançou.

No domínio financeiro, itens não aprovados são marcados como rejeitados, mantendo o histórico do orçamento. Se o pagamento não for confirmado, a OS não deve avançar para execução como se estivesse paga; ela permanece aguardando regularização do pagamento.

Confiabilidade e rastreabilidade

As mensagens carregam um `correlationId`, que permite rastrear a mesma Ordem de Serviço entre serviços, logs e eventos. Como o Amazon SQS pode entregar mensagens repetidas ou fora de ordem, os consumidores precisam ser idempotentes: uma mensagem já aplicada não deve gerar uma nova reserva, um novo snapshot ou uma transição de status indevida.

Além disso, o serviço de execução publica eventos após a confirmação da transação local, reduzindo o risco de publicar um evento relacionado a uma gravação que sofreu rollback. As DLQs permitem isolar falhas e reprocessá-las de maneira controlada.

Portanto, a coreografia foi escolhida por promover baixo acoplamento, escalabilidade e autonomia dos microsserviços. O custo dessa decisão é a necessidade de tratar consistência eventual, idempotência, observabilidade e ações compensatórias, todos atendidos no desenho por meio de eventos, filas, `correlationId`, transições protegidas de estado e DLQs.